

Тема 11 Триггеры и хранимые процедуры

Триггер - это специальный тип хранимой процедуры, которая автоматически вызывается, когда данные в определенной таблице добавляются, удаляются или изменяются с помощью SQL-предложений INSERT, DELETE или UPDATE. В зависимости от того, какое из событий, связанных с изменением данных, инициирует запуск триггера, он называется insert trigger, delete trigger или update trigger.

Некоторые базы данных позволяют создавать разные триггеры для выполнения до и после вставки, удаления или изменения записей. Обратите также внимание на то, что большинство СУБД позволяют создавать несколько триггеров для одного и того же события. В этом случае следует определить порядок, в котором они будут выполняться.

Триггеры часто используются для поддержки ссылочной целостности, каскадного изменения или удаления данных (то есть изменения или удаления всех дочерних записей вместе с родительской), архивирования удаленных или измененных данных, отслеживания изменений или вызова пользовательских или системных хранимых процедур.

Триггеры могут опосредованно вызывать другие триггеры. Например, если триггер, выполняющийся в данный момент, содержит код, модифицирующий другую таблицу, имеющую собственный триггер, последний будет запущен. В свою очередь, он может вызвать следующий триггер и так далее.

Как и многие другие объекты баз данных, триггеры создаются с помощью предложения CREATE. Чтобы изменить триггер, мы используем предложение ALTER, а для удаления триггера - предложение DROP. В последующих разделах мы рассмотрим синтаксис этих предложений и затем создадим триггер, который может быть использован для записи сведений о модификациях данных.

В предложении CREATE TRIGGER можно использовать две специальные таблицы.

Например, таблицы deleted и inserted имеют ту же структуру, что и таблица, для которой определен триггер, и содержат старое и новое значения записей, измененных пользователем.

Например, мы можем использовать следующее SQL-предложение для поиска удаленных записей:

```
SELECT * FROM deleted
```

В таблице ниже показано содержимое таблиц deleted и inserted для всех возможных изменений данных.

Изменение	Содержимое таблицы inserted	Содержимое таблицы deleted
INSERT	Добавленные записи	-
DELETE	-	Удаленные записи
UPDATE	Новые значения для модифицированных записей	Старые значения для модифицированных записей

Для изменения имеющегося триггера следует использовать предложение **ALTER TRIGGER**.

Данное предложение имеет тот же синтаксис, что и предложение **CREATE TRIGGER**, которое мы обсудим выше. Это предложение применяется для изменения текста имеющегося триггера. Например, если мы решили, что какой-либо триггер, определенный для всех трех типов модификаций данных, теперь не должен выполняться, когда записи удаляются, нам следует использовать тот же самый код, с помощью которого мы создавали триггер, но удалить ключевое слово **UPDATE** в предложении **FOR**.

Если триггер больше не нужен, мы можем удалить его из базы данных. Для этого применяется предложение **DROP TRIGGER**.

Это предложение удаляет триггер из базы данных. Используя это предложение, нужно указывать имя удаляемого триггера. Обратите внимание на то, что триггер автоматически удаляется при удалении таблицы, для которой он создан.

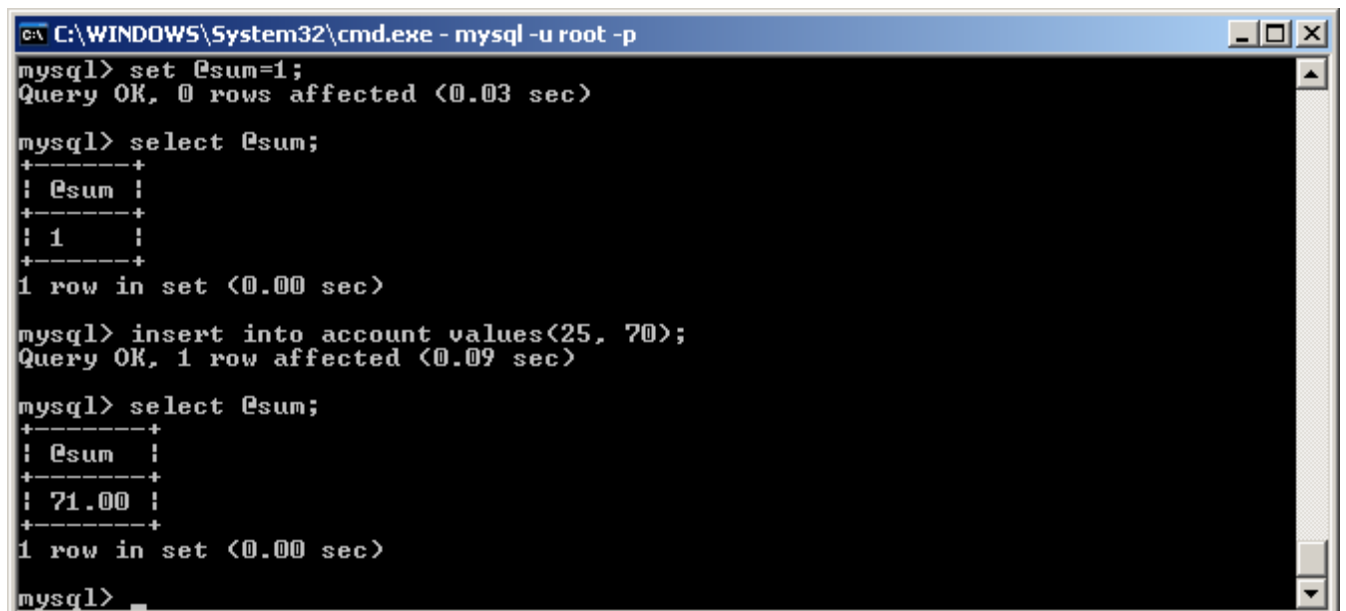
Создание и использование триггеров

Например, нижеприведенный код создает таблицу и **INSERT** триггер. Триггер суммирует значения, вставляемые в один из столбцов таблицы.

```
mysql> CREATE TABLE account (acct_num INT, amount DECIMAL(10,2));
Query OK, 0 rows affected (0.03 sec)
mysql> CREATE TRIGGER ins_sum BEFORE INSERT ON account
-> FOR EACH ROW SET @sum = @sum + NEW.amount;
Query OK, 0 rows affected (0.06 sec)
```

Объявим переменную **sum** и присвоим ей значение 1. После этого при каждой вставке в таблицу **account** значение этой переменной будет увеличиваться согласно вставляемой части.

Если значение переменной не инициализировано, то триггер работать не будет.



```
C:\WINDOWS\System32\cmd.exe - mysql -u root -p
mysql> set @sum=1;
Query OK, 0 rows affected (0.03 sec)

mysql> select @sum;
+-----+
| @sum |
+-----+
| 1     |
+-----+
1 row in set (0.00 sec)

mysql> insert into account values(25, 70);
Query OK, 1 row affected (0.09 sec)

mysql> select @sum;
+-----+
| @sum |
+-----+
| 71.00 |
+-----+
1 row in set (0.00 sec)

mysql> _
```

Синтаксис создания триггера

CREATE

[DEFINER = { имя_пользователя | CURRENT_USER }]

TRIGGER имя_триггера время_триггера событие_срабатывания_триггера

ON имя_таблицы FOR EACH ROW

выражение_выполняемое_при_срабатывании_триггера

где:

время_триггера - определяет время свершения действия триггера. BEFORE означает, что триггер выполнится до завершения события срабатывания триггера, а AFTER означает, что после. Например, при вставке записей (см. пример выше) наш триггер срабатывал до фактической вставки записи и вычислял сумму. Такой вариант уместен при предварительном вычислении каких-то дополнительных полей в таблице или параллельной вставке в другую таблицу.

событие_срабатывания_триггера - здесь четко обозначается, при каком событии выполняется триггер.

- INSERT: т.е. при операциях вставки или аналогичных ей выражениях (INSERT, LOAD DATA, и REPLACE)

- UPDATE: когда сущность (строка) модифицирована

- DELETE: когда запись удаляется (запросы, содержащие выражения DELETE и/или REPLACE)

Обзор хранимых процедур

Хранимая процедура (хранимая процедура) - это набор операторов SQL для выполнения определенной функции, хранящейся в базе данных, после первой компиляции она вызывается снова без повторной компиляции. Пользователь указывает имя хранимой процедуры и дает параметры (если Хранимая процедура принимает параметры) для ее выполнения. Подобно методу в Java, его можно вызывать несколько раз после определения.

Хранимые процедуры в SQL – это аналог функций в других языках программирования. Хранимые процедуры могут выполнять действия над данными автоматически: вывод данных, удаление, изменение.

Особенностью процедур является то, что есть возможность передавать аргументы, (так же как и функциям в других языках) и выводить различные данные в зависимости от аргумента. Также, процедура является сущностью SQL, которую создают один раз, а затем вызывают, передавая аргументы.

Хранимая процедура имеет следующие преимущества

1. Обычно хранимые процедуры помогают повысить производительность приложений. При создании хранимая процедура компилируется и сохраняется в базе данных.

2. Может использоваться для улучшения возможности повторного использования кода и упрощения операций.

3. Сохраненная программа безопасна.

Хранимые процедуры в СУБД MySQL можно создать несколькими способами. Опишем 2 способа через утилиту **phpmyadmin**:

- Через консоль SQL запросов

Также как и все запросы SELECT, INSERT и т.д код создания хранимых процедур можно прописывать на вкладке SQL.

- Через специальную вкладку процедуры

Второй способ возможен тогда, когда мы выбрали определенную БД, и конкретно для нее появляется вкладка процедур.

Синтаксическая структура для создания хранимых процедур

```
DROP PROCEDURE IF EXISTS [Имя хранимой процедуры];

DELIMITER [конечный тег]

CREATE PROCEDURE [Имя хранимой процедуры]([Параметр 1], [Параметр 2] ...)
BEGIN
    [Тело хранимой процедуры(Набор допустимых операторов SQL)]
END [Конечный тег]
DELIMITER ;
```

Когда хранимая процедура имеет один и только один оператор SQL, BEGIN с END Может быть опущено. Параметры в процессе хранения можно разделить на 3 части, а именно [Режим параметров] [Имя параметра] [Тип параметра] Такие как IN name VARCHAR[20].

Режимы параметров также делятся на 3 типа, а именно:

IN: Входные параметры

OUT: Может использоваться как параметр возвращаемого значения

INOUT: Может использоваться как входной параметр или как параметр возвращаемого значения

DELIMITER [имя хранимой процедуры] Он не имеет ничего общего с синтаксисом хранимой процедуры и используется для обозначения конца хранимой процедуры. Последняя команда (DELIMITER ;) Измените разделитель обратно на точку с запятой (;).

Простое использование хранимых процедур. Создадим тестовую таблицу

```
DROP TABLE IF EXISTS `t_user`;

CREATE TABLE `t_user` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(20) NOT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8;
```

Создадим хранимую процедуру (чтобы **t_user** Вставим два фрагмента данных в таблицу данных)

```
DROP PROCEDURE IF EXISTS myp1;
DELIMITER $
CREATE PROCEDURE myp1()
BEGIN
    INSERT INTO t_user VALUES(NULL, 'Jas'),(NULL, 'Joy');
END $
DELIMITER ;
```

Вызов хранимой процедуры для использования

CALL [имя хранимой процедуры] ([параметр 1], [параметр 2] ...);

Исключить использование хранимых процедур

DROP PROCEDURE IF EXISTS [имя хранимой процедуры];

Вызовем хранимую процедуру и посмотрим результаты.

```
mysql> CALL myp1();
Query OK, 2 rows affected (0.11 sec)

mysql> SELECT * FROM t_user;
+----+-----+
| id | name |
+----+-----+
|  1 | Jas  |
|  2 | Joy  |
+----+-----+
2 rows in set (0.00 sec)
```

Создание **IN** Сохраненная процедура для параметров режима

Создадим хранимую процедуру (согласно входящей **id** Получим логин)

```
DROP PROCEDURE IF EXISTS myp2;
DELIMITER $
CREATE PROCEDURE myp2(IN userId INT)
BEGIN
    SELECT name FROM t_user WHERE id = userId;
END $
DELIMITER ;
```

Вызов хранимой процедуры и просмотр результатов

```
mysql> CALL myp2(1);
```

id	name
1	Jas

```
1 row in set (0.00 sec)
```

Создание **OUT** Сохраненная процедура для параметров режима

Создайте хранимую процедуру (согласно входящей **id** Получить имя пользователя, назначенное выходной переменной)

```
DROP PROCEDURE IF EXISTS myp3;
DELIMITER $
CREATE PROCEDURE myp3(IN userId INT, OUT username VARCHAR(20))
BEGIN
    SELECT name INTO username      # Назначьте запрашиваемое имя пользователя
    FROM t_user WHERE id = userId;
END $
DELIMITER ;
```

Создание **INOUT** Сохраненная процедура для параметров режима

Создадим хранимую процедуру (передать целочисленный параметр и вернуть после увеличения значения в 2 раза)

```
DROP PROCEDURE IF EXISTS myp4;
DELIMITER $
CREATE PROCEDURE myp4(INOUT a INT)
BEGIN
    SET a = a * 2;
END $
DELIMITER ;
```

Чтобы создать процедуру необходимо воспользоваться оператором **CREATE PROCEDURE**. После оператора следует указать имя процедуры, а затем в круглых скобках аргументы, если они имеются, вместе с указанием типа данных каждого аргумента. Примерно это будет выглядеть так:

```
CREATE PROCEDURE name_procedure (arg1 datatype, arg2
datatype,...)
```

Например, напомним процедуру, которая будет показывать список покупателей из города, который мы передадим в качестве аргумента.

```
DELIMITER //
```

```
CREATE PROCEDURE get_customers (city_arg VARCHAR(45))
```

```
BEGIN
```

```
    SELECT cname, city
```

```
    FROM customers
```

```
    WHERE city = city_arg;
```

```
END //
```

```
DELIMITER ;
```

Здесь используем первый способ создания процедуры — через консоль SQL для нашей базы данных.

Поясним некоторые моменты приведенного кода:

- По умолчанию окончанием запроса служит точка с запятой. Оператор **DELIMITER** задает новый разделитель, то есть новый знак окончания запроса. Это необходимо для того, чтобы запрос создания процедуры заканчивался после ключевого слова **END**. В противном случае, запрос закончится на точке с запятой, после основного тела процедуры.

- Процедура должна начинаться с **BEGIN** и заканчиваться **END**. Внутри процедуры используется знакомый запрос **SELECT**, который сравнивает город со значением аргумента.

Итак, процедура создана и хранится отдельно от таблиц или представлений.

Для использования процедуры, ее следует вызвать и передать входные аргументы, если они требуются. Вызов хранимой процедуры в SQL производится с помощью оператора **CALL**. Вызов созданной выше процедуры будет произведен следующим образом:

```
CALL get_customers ("Москва");
```

Хранимые процедуры это мощный инструмент. И в SQL хранимые процедуры также могут использовать различные операторы, вложенные запросы и т.д.

Также, существуют еще особенности хранимых процедур. В хранимых процедурах есть возможность использовать условные операторы и циклы, такие как **IF-ELSE**, **CASE** и **WHILE**.

Приведем пример использования конструкции **CASE**: создать хранимую процедуру, которая выведет продавцов, оформивших заказы, по диапазонам. Если аргумент имеет значение «Маленькая суммы», то диапазон продаж от 0 до 1000, «Средние суммы» — от 1000 до 1500, «Большие суммы» — свыше 1500.

```

DELIMITER //
CREATE PROCEDURE get_salespeople(str VARCHAR(45))
BEGIN
  CASE str
  WHEN "Маленькие суммы"
  THEN
    SELECT sname as "Имя продавца", SUM(amt) as "Суммарные продажи"
    FROM salespeople
    INNER JOIN orders on salespeople.snum = orders.snum
    group by sname
    HAVING SUM(amt) < 1000;
  WHEN "Средние суммы"
  THEN
    SELECT sname as "Имя продавца", SUM(amt) as "Суммарные продажи"
    FROM salespeople
    INNER JOIN orders on salespeople.snum = orders.snum
    group by sname
    HAVING SUM(amt) >= 1000 and SUM(amt) < 1500;
  WHEN "Большие суммы"
  THEN
    SELECT sname as "Имя продавца", SUM(amt) as "Суммарные
продажи"
    FROM salespeople
    INNER JOIN orders on salespeople.snum = orders.snum
    group by sname
    HAVING SUM(amt) >= 1500;
  END CASE;
END //
DELIMITER;

```

Вызвать созданную процедуру можно с тремя различными аргументами:

```

CALL get_salespeople("Маленькие суммы");
CALL get_salespeople("Средние суммы");
CALL get_salespeople("Большие суммы");

```

Соответственно, в выводе будут разные данные в зависимости от аргумента.

Примеры

1. Создать хранимую процедуру, которая определяет лучшего продавца по суммарному значению продаж за период дат, полученных в аргументах.

```

DELIMITER //
CREATE PROCEDURE best_salespeople (d1 DATE, d2 DATE)
BEGIN

```



```

SELECT sname, SUM(amt) AS "Суммарные продажи"
FROM salespeople
INNER JOIN orders ON orders.snum = salespeople.snum
and odate BETWEEN d1 and d2
GROUP BY sname
ORDER BY SUM(amt) DESC
LIMIT 1;
END //
DELIMITER ;
CALL best_salespeople ('2010-01-01', '2019-01-01');

```

2. Создать хранимую процедуру, которая отобразит информацию о совершенных заказах, стоимость которых выше числа, заданного в аргументе.

```

DELIMITER //
CREATE PROCEDURE info_orders (arg int)
BEGIN
    SELECT onum, amt, odate
    FROM orders
    WHERE amt > arg;
END //
DELIMITER ;

```